```
#importing required libraries

import numpy as np
import pandas as pd
```

1.Load the data

```
#importing dataset

df=pd.read_csv('/content/Credit Score Classification Dataset.csv')
df
```

	Age	Gender	Income	Education	Marital Status	Number of Children	Home Ownership	Credit Score
0	25	Female	50000.0	Bachelor's Degree	Single	0	Rented	High
1	30	Male	100000.0	Master's Degree	Married	2	Owned	High
2	35	Female	75000.0	Doctorate	Married	1	Owned	High
3	40	Male	125000.0	High School Diploma	Single	0	Owned	High
4	45	Female	100000.0	Bachelor's Degree	Married	3	Owned	High
...	...	...	...	...	...	...	...	...
159	29	Female	27500.0	High School Diploma	Single	0	Rented	Low
160	34	Male	47500.0	Associate's Degree	Single	0	Rented	Average
161	39	Female	62500.0	Bachelor's Degree	Married	2	Owned	High
162	44	Male	87500.0	Master's Degree	Single	0	Owned	High
163	49	Female	77500.0	Doctorate	Married	1	Owned	High

164 rows × 8 columns

```
#printing first 5 values

df.head()
```

	Age	Gender	Income	Education	Marital Status	Number of Children	Home Ownership	Credit Score
0	25	Female	50000.0	Bachelor's Degree	Single	0	Rented	High
1	30	Male	100000.0	Master's Degree	Married	2	Owned	High
2	35	Female	75000.0	Doctorate	Married	1	Owned	High
3	40	Male	125000.0	High School Diploma	Single	0	Owned	High
4	45	Female	100000.0	Bachelor's Degree	Married	3	Owned	High

```
#printing last 5 values

df.tail()
```

	Age	Gender	Income	Education	Marital Status	Number of Children	Home Ownership	Credit Score
159	29	Female	27500.0	High School Diploma	Single	0	Rented	Low
160	34	Male	47500.0	Associate's Degree	Single	0	Rented	Average
161	39	Female	62500.0	Bachelor's Degree	Married	2	Owned	High
162	44	Male	87500.0	Master's Degree	Single	0	Owned	High
163	49	Female	77500.0	Doctorate	Married	1	Owned	High

```
#number of rows and columns

df.shape
```

```
(164, 8)
```

```
df['Credit Score'].value_counts()
```

```
High       113
Average     36
Low         15
Name: Credit Score, dtype: int64
```

```
#=====>it's an imbalanced data
```

2.Dataset cleaning

```
#finding missing values

df.isna().sum()
```

```
Age                  0
Gender               6
Income               9
Education            0
Marital Status       0
Number of Children   0
Home Ownership       0
Credit Score         0
dtype: int64
```

```
#filling the missing values

df['Gender']=df['Gender'].fillna(df['Gender'].mode()[0])
df['Income']=df['Income'].fillna(df['Income'].mean())
df.isna().sum()
```

```
Age                   0
Gender                0
Income                0
Education             0
Marital Status        0
Number of Children    0
Home Ownership        0
Credit Score          0
dtype: int64
```

```
#printing datatypes

df.dtypes
```

```
Age                     int64
Gender                 object
Income                float64
Education              object
Marital Status         object
Number of Children      int64
Home Ownership         object
Credit Score           object
dtype: object
```

3.Preprocessing

```
#Encoding

from sklearn.preprocessing import LabelEncoder,StandardScaler
le=LabelEncoder()
for i in df.columns:
  if df[i].dtype=='object':
    df[i]=le.fit_transform(df[i])
```

```
#high=1  avrg=0 low=2
```

```
#printing datatypes

df.dtypes
```

```
Age                     int64
Gender                  int64
Income                float64
Education               int64
Marital Status          int64
Number of Children      int64
Home Ownership          int64
Credit Score            int64
dtype: object
```

```
#input label
```

```
x=df.iloc[:,:-1].values
x
```

```
array([[2.50e+01, 0.00e+00, 5.00e+04, ..., 1.00e+00, 0.00e+00, 1.00e+00],
       [3.00e+01, 1.00e+00, 1.00e+05, ..., 0.00e+00, 2.00e+00, 0.00e+00],
       [3.50e+01, 0.00e+00, 7.50e+04, ..., 0.00e+00, 1.00e+00, 0.00e+00],
       ...,
       [3.90e+01, 0.00e+00, 6.25e+04, ..., 0.00e+00, 2.00e+00, 0.00e+00],
       [4.40e+01, 1.00e+00, 8.75e+04, ..., 1.00e+00, 0.00e+00, 0.00e+00],
       [4.90e+01, 0.00e+00, 7.75e+04, ..., 0.00e+00, 1.00e+00, 0.00e+00]])
```

```
#output label

y=df.iloc[:,-1].values
y
```

```
array([1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 1, 2, 0, 1, 1,
       1, 1, 2, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 2, 0, 1,
       1, 1, 1, 2, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1,
       1, 0, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 1, 2, 0, 1,
       1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1,
       1, 2, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0,
       1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 2, 0, 1, 1, 1, 1, 2,
       0, 1, 1, 1, 1, 2, 0, 1, 1, 1])
```

```
#splitting the data for training and testing

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.30,random_state=42)
x_train
```

```
array([[4.50000000e+01, 0.00000000e+00, 1.15000000e+05, 1.00000000e+00,
        0.00000000e+00, 3.00000000e+00, 0.00000000e+00],
       [5.10000000e+01, 1.00000000e+00, 1.35000000e+05, 1.00000000e+00,
        0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [4.50000000e+01, 0.00000000e+00, 1.10000000e+05, 1.00000000e+00,
        0.00000000e+00, 3.00000000e+00, 0.00000000e+00],
       [3.40000000e+01, 1.00000000e+00, 4.75000000e+04, 0.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
       [2.60000000e+01, 0.00000000e+00, 4.00000000e+04, 0.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
       [4.40000000e+01, 1.00000000e+00, 7.50000000e+04, 4.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [4.20000000e+01, 1.00000000e+00, 1.05000000e+05, 4.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [3.10000000e+01, 1.00000000e+00, 6.50000000e+04, 1.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
       [4.10000000e+01, 1.00000000e+00, 1.10000000e+05, 2.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [2.70000000e+01, 0.00000000e+00, 3.75000000e+04, 3.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
       [4.50000000e+01, 0.00000000e+00, 1.00000000e+05, 1.00000000e+00,
        0.00000000e+00, 3.00000000e+00, 0.00000000e+00],
       [4.00000000e+01, 1.00000000e+00, 1.30000000e+05, 3.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [4.00000000e+01, 1.00000000e+00, 1.30000000e+05, 3.00000000e+00,
```

```
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[4.10000000e+01, 1.00000000e+00, 1.10000000e+05, 2.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[4.70000000e+01, 0.00000000e+00, 8.39516129e+04, 2.00000000e+00,
 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
[3.10000000e+01, 1.00000000e+00, 6.75000000e+04, 1.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[4.60000000e+01, 0.00000000e+00, 9.00000000e+04, 3.00000000e+00,
 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
[4.00000000e+01, 1.00000000e+00, 1.42500000e+05, 3.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[2.80000000e+01, 0.00000000e+00, 8.39516129e+04, 0.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[2.50000000e+01, 0.00000000e+00, 5.00000000e+04, 1.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[2.50000000e+01, 0.00000000e+00, 5.75000000e+04, 1.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[3.00000000e+01, 1.00000000e+00, 1.12500000e+05, 4.00000000e+00,
 0.00000000e+00, 2.00000000e+00, 0.00000000e+00],
[4.00000000e+01, 1.00000000e+00, 1.42500000e+05, 3.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[4.20000000e+01, 1.00000000e+00, 1.00000000e+05, 4.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[5.10000000e+01, 1.00000000e+00, 1.40000000e+05, 1.00000000e+00,
 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[4.90000000e+01, 0.00000000e+00, 6.50000000e+04, 2.00000000e+00,
 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
[5.10000000e+01, 1.00000000e+00, 1.40000000e+05, 1.00000000e+00,
 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[4.30000000e+01, 1.00000000e+00, 8.39516129e+04, 4.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[3.40000000e+01, 1.00000000e+00, 4.50000000e+04, 0.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
```

```
x_test
```

```
array([[2.50000000e+01, 0.00000000e+00, 5.50000000e+04, 1.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
       [4.80000000e+01, 0.00000000e+00, 8.39516129e+04, 2.00000000e+00,
        0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
       [2.60000000e+01, 0.00000000e+00, 5.50000000e+04, 1.00000000e+00,
        0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
       [3.90000000e+01, 0.00000000e+00, 6.25000000e+04, 1.00000000e+00,
        0.00000000e+00, 2.00000000e+00, 0.00000000e+00],
       [3.50000000e+01, 0.00000000e+00, 9.00000000e+04, 2.00000000e+00,
        0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
       [2.50000000e+01, 0.00000000e+00, 5.50000000e+04, 1.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
       [4.80000000e+01, 0.00000000e+00, 8.25000000e+04, 2.00000000e+00,
        0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
       [4.80000000e+01, 0.00000000e+00, 8.25000000e+04, 2.00000000e+00,
        0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
       [3.60000000e+01, 0.00000000e+00, 9.50000000e+04, 4.00000000e+00,
        0.00000000e+00, 2.00000000e+00, 0.00000000e+00],
       [4.60000000e+01, 0.00000000e+00, 9.50000000e+04, 3.00000000e+00,
        0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
       [3.30000000e+01, 1.00000000e+00, 5.00000000e+04, 3.00000000e+00,
        1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
       [2.90000000e+01, 0.00000000e+00, 2.75000000e+04, 3.00000000e+00,
```

```
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[4.20000000e+01, 1.00000000e+00, 9.50000000e+04, 4.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[5.00000000e+01, 1.00000000e+00, 1.60000000e+05, 4.00000000e+00,
 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[2.90000000e+01, 0.00000000e+00, 2.50000000e+04, 3.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[3.00000000e+01, 1.00000000e+00, 1.05000000e+05, 4.00000000e+00,
 0.00000000e+00, 2.00000000e+00, 0.00000000e+00],
[3.20000000e+01, 1.00000000e+00, 8.50000000e+04, 4.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[2.70000000e+01, 0.00000000e+00, 3.75000000e+04, 3.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[5.20000000e+01, 0.00000000e+00, 1.30000000e+05, 3.00000000e+00,
 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[4.70000000e+01, 0.00000000e+00, 8.50000000e+04, 2.00000000e+00,
 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
[3.70000000e+01, 0.00000000e+00, 7.50000000e+04, 1.00000000e+00,
 0.00000000e+00, 2.00000000e+00, 0.00000000e+00],
[2.80000000e+01, 0.00000000e+00, 3.00000000e+04, 0.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[2.70000000e+01, 0.00000000e+00, 3.50000000e+04, 3.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 1.00000000e+00],
[4.10000000e+01, 1.00000000e+00, 1.05000000e+05, 2.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[3.50000000e+01, 0.00000000e+00, 8.00000000e+04, 2.00000000e+00,
 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
[3.80000000e+01, 0.00000000e+00, 6.75000000e+04, 1.00000000e+00,
 0.00000000e+00, 2.00000000e+00, 0.00000000e+00],
[5.00000000e+01, 1.00000000e+00, 1.62500000e+05, 4.00000000e+00,
 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[4.40000000e+01, 1.00000000e+00, 8.75000000e+04, 4.00000000e+00,
 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
[3.40000000e+01, 1.00000000e+00, 1.05000000e+05, 1.00000000e+00,
```

```
y_train
```

```
array([1, 1, 1, 0, 0, 1, 1, 0, 1, 2, 1, 1, 1, 1, 1, 0, 1, 1, 2, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 2, 1, 0, 1, 2, 1, 2, 1, 0, 2, 1, 0,
       1, 0, 1, 1, 0, 1, 2, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 2, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1,
       0, 1, 1, 1])
```

```
y_test
```

```
array([0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 2, 1, 1, 2, 1, 1, 2, 1, 1, 1, 2,
       2, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 2, 1, 0, 2, 0,
       0, 0, 1, 1, 1, 0])
```

```
#Normalization

scaler=StandardScaler()
scaler.fit(x_train)
x_train=scaler.transform(x_train)
x_test=scaler.transform(x_test)
x_train
```

```
array([[ 0.77593283, -1.01770049,  0.94515333, -0.73865505, -0.98260737,
         2.66391337, -0.66561098],
       [ 1.49704432,  0.98260737,  1.59465735, -0.73865505, -0.98260737,
        -0.69792564, -0.66561098],
       [ 0.77593283, -1.01770049,  0.78277732, -0.73865505, -0.98260737,
         2.66391337, -0.66561098],
       [-0.5461049 ,  0.98260737, -1.24692275, -1.45227095,  1.01770049,
        -0.69792564,  1.50237907],
       [-1.50758688, -1.01770049, -1.49048675, -1.45227095,  1.01770049,
        -0.69792564,  1.50237907],
       [ 0.65574758,  0.98260737, -0.35385472,  1.40219264,  1.01770049,
        -0.69792564, -0.66561098],
       [ 0.41537709,  0.98260737,  0.62040132,  1.40219264,  1.01770049,
        -0.69792564, -0.66561098],
       [-0.90666064,  0.98260737, -0.67860673, -0.73865505,  1.01770049,
        -0.69792564,  1.50237907],
       [ 0.29519184,  0.98260737,  0.78277732, -0.02503915,  1.01770049,
        -0.69792564, -0.66561098],
       [-1.38740164, -1.01770049, -1.57167476,  0.68857674,  1.01770049,
        -0.69792564,  1.50237907],
       [ 0.77593283, -1.01770049,  0.45802531, -0.73865505, -0.98260737,
         2.66391337, -0.66561098],
       [ 0.17500659,  0.98260737,  1.43228134,  0.68857674,  1.01770049,
        -0.69792564, -0.66561098],
       [ 0.17500659,  0.98260737,  1.43228134,  0.68857674,  1.01770049,
        -0.69792564, -0.66561098],
       [ 0.29519184,  0.98260737,  0.78277732, -0.02503915,  1.01770049,
        -0.69792564, -0.66561098],
       [ 1.01630333, -1.01770049, -0.06314929, -0.02503915, -0.98260737,
         0.42268736, -0.66561098],
       [-0.90666064,  0.98260737, -0.59741872, -0.73865505,  1.01770049,
        -0.69792564,  1.50237907],
       [ 0.89611808, -1.01770049,  0.1332733 ,  0.68857674, -0.98260737,
         0.42268736, -0.66561098],
       [ 0.17500659,  0.98260737,  1.83822136,  0.68857674,  1.01770049,
        -0.69792564, -0.66561098],
       [-1.26721639, -1.01770049, -0.06314929, -1.45227095,  1.01770049,
        -0.69792564,  1.50237907],
       [-1.62777213, -1.01770049, -1.16573474, -0.73865505,  1.01770049,
        -0.69792564,  1.50237907],
       [-1.62777213, -1.01770049, -0.92217074, -0.73865505,  1.01770049,
        -0.69792564,  1.50237907],
       [-1.02684589,  0.98260737,  0.86396532,  1.40219264, -0.98260737,
         1.54330037, -0.66561098],
       [ 0.17500659,  0.98260737,  1.83822136,  0.68857674,  1.01770049,
        -0.69792564, -0.66561098],
       [ 0.41537709,  0.98260737,  0.45802531,  1.40219264,  1.01770049,
        -0.69792564, -0.66561098],
       [ 1.49704432,  0.98260737,  1.75703335, -0.73865505, -0.98260737,
        -0.69792564, -0.66561098],
       [ 1.25667382, -1.01770049, -0.67860673, -0.02503915, -0.98260737,
```

```
  0.42268736, -0.66561098],
 [ 1.49704432,  0.98260737,  1.75703335, -0.73865505, -0.98260737,
 -0.69792564, -0.66561098],
 [ 0.53556233,  0.98260737, -0.06314929,  1.40219264,  1.01770049,
 -0.69792564, -0.66561098],
 [-0.5461049 ,  0.98260737, -1.32811075, -1.45227095,  1.01770049,
 -0.69792564,  1.50237907],
```

K-Nearest Neighbour:

4.Model creation

```
from sklearn.neighbors import KNeighborsClassifier
knn=KNeighborsClassifier(n_neighbors=7)
knn.fit(x_train,y_train)
y_pred_knn=knn.predict(x_test)
y_pred_knn
```

```
array([0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 2, 1, 1, 2, 1, 0, 2, 1, 1, 1, 0,
       2, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0,
       1, 0, 1, 1, 1, 0])
```

```
y_test
```

```
array([0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 2, 1, 1, 2, 1, 1, 2, 1, 1, 1, 2,
       2, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 2, 1, 0, 2, 0,
       0, 0, 1, 1, 1, 0])
```

5.Performance evaluation

```
from sklearn.metrics import confusion_matrix,accuracy_score,ConfusionMatrixDisplay,classification_report

    #accuracy score

score=accuracy_score(y_test,y_pred_knn)
score
```

```
0.86
```

```
    #classification report

print(classification_report(y_test,y_pred_knn))
```

```
              precision    recall  f1-score   support

           0       0.64      0.82      0.72        11
           1       0.94      0.94      0.94        32
           2       1.00      0.57      0.73         7

    accuracy                           0.86        50
   macro avg       0.86      0.78      0.79        50
```

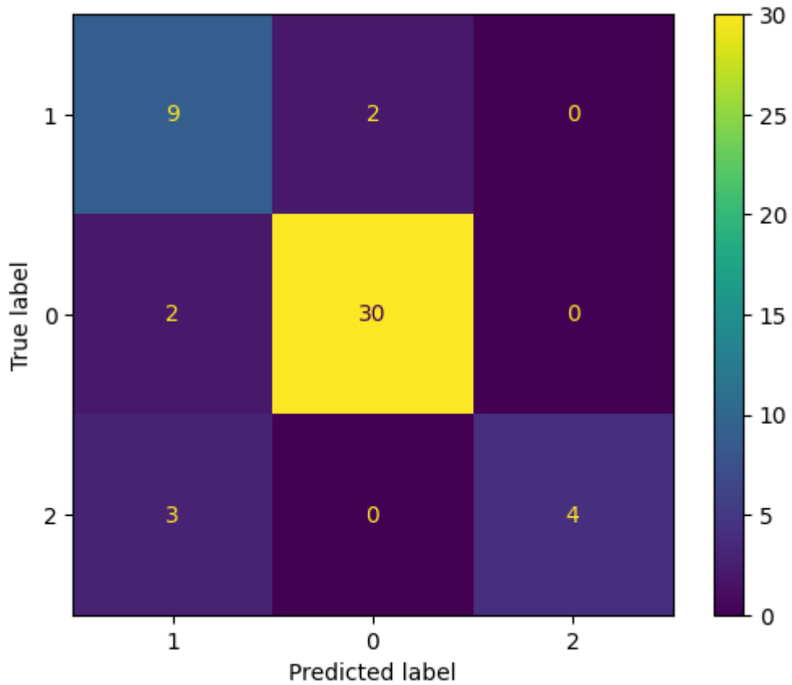
```
weighted avg       0.88      0.86      0.86        50
```

```
#confusion matrix

matr=confusion_matrix(y_test,y_pred_knn)
matr
```

```
array([[ 9,  2,  0],
       [ 2, 30,  0],
       [ 3,  0,  4]])
```

```
#confusion matrix display

label=[1,0,2]
cmd=ConfusionMatrixDisplay(matr,display_labels=label)
cmd.plot()
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7ebcdf70afe0>
```



Naive Bayes

4.Model creation

```
from sklearn.naive_bayes import BernoulliNB
model=BernoulliNB()
```

```
model.fit(x_train,y_train)
y_pred_nb=model.predict(x_test)
y_pred_nb
```

```
array([0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 2, 1, 1, 2, 1, 0, 2, 1, 1, 1, 0,
       2, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 2, 0,
       1, 0, 1, 1, 1, 0])
```

```
y_test
```

```
array([0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 2, 1, 1, 2, 1, 1, 2, 1, 1, 1, 2,
       2, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 2, 1, 0, 2, 0,
       0, 0, 1, 1, 1, 0])
```

5.Performance evaluation

```
    #accuracy score

score=accuracy_score(y_test,y_pred_nb)
score
```

```
0.9
```

```
    #classification report

print(classification_report(y_test,y_pred_nb))
```

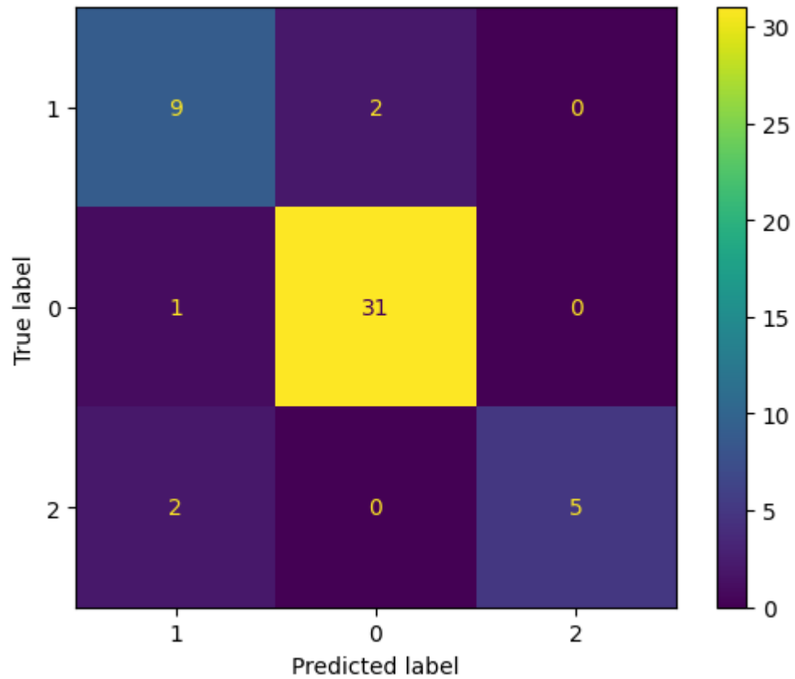
```
              precision    recall  f1-score   support

           0       0.75      0.82      0.78        11
           1       0.94      0.97      0.95        32
           2       1.00      0.71      0.83         7

    accuracy                           0.90        50
   macro avg       0.90      0.83      0.86        50
weighted avg       0.91      0.90      0.90        50
```

```
    #confusion matrix

matr=confusion_matrix(y_test,y_pred_nb)
matr
```

```
array([[ 9,  2,  0],
       [ 1, 31,  0],
       [ 2,  0,  5]])
```

```
    #confusion matrix display

label=[1,0,2]
cmd=ConfusionMatrixDisplay(matr,display_labels=label)
cmd.plot()
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7ebcde7afc70>
```



SVM

4.Model creation

```
from sklearn.svm import SVC
model1=SVC()
model1.fit(x_train,y_train)
y_pred_svm=model1.predict(x_test)
y_pred_svm
```

```
array([0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 2, 1, 1, 2, 1, 0, 2, 1, 1, 1, 0,
       2, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0,
       1, 0, 1, 1, 1, 0])
```

5.Performance evaluation

```
#accuracy score

score=accuracy_score(y_test,y_pred_svm)
score
```

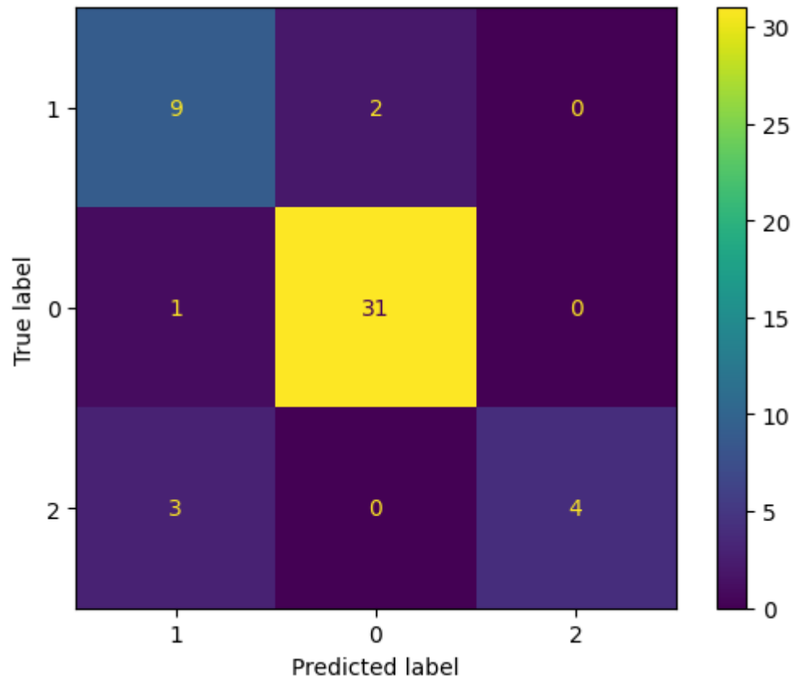
```
0.88
```

```
    #classification report

print(classification_report(y_test,y_pred_svm))
```

```
              precision    recall  f1-score   support

           0       0.69      0.82      0.75        11
           1       0.94      0.97      0.95        32
           2       1.00      0.57      0.73         7

    accuracy                           0.88        50
   macro avg       0.88      0.79      0.81        50
weighted avg       0.89      0.88      0.88        50
```

```
     #confusion matrix

matr=confusion_matrix(y_test,y_pred_svm)
matr
```

```
array([[ 9,  2,  0],
       [ 1, 31,  0],
       [ 3,  0,  4]])
```

```
    #confusion matrix display

label=[1,0,2]
cmd=ConfusionMatrixDisplay(matr,display_labels=label)
cmd.plot()
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7ebcde200820>
```